

Segundo Trabajo Grupal



Diego Castro, Gonzalo Muñoz, Juan López, Malena González

Descripción del problema

El trabajo consiste en crear un sistema de gestión bibliotecaria para la UCU. El sistema debe permitir registrar libros (con su ISBN, título, autor, género y cantidad de ejemplares), gestionar usuarios (socios de la biblioteca) y administrar préstamos y devoluciones. Toda la información debe persistir en un archivo para que el estado del sistema se mantenga entre ejecuciones.

El sistema tiene que resolver los siguientes problemas:

- Control del inventario de libros y sus ejemplares disponibles.
- Registro y baja de socios de la biblioteca.
- Registro de préstamos con fecha límite automática (7 días desde el préstamo).
- Registro de devoluciones y actualización del inventario.
- Detección de préstamos vencidos.
- Historial de préstamos por usuario.
- Recomendación de libros según el historial de lectura (función bonus).

Descripción de las funciones implementadas

agregar_libro(catalogo, isbn, titulo, autor, genero, ejemplares)

Verifica que el ISBN no exista en el catálogo (raise Exception si ya existe). Construye un diccionario Libro con ejemplares_totales y ejemplares_disponibles iguales al parámetro ejemplares, lo inserta en el catálogo y persiste los cambios.

eliminar_libro(catalogo, prestamos, isbn)

Verifica que el ISBN exista (raise si no existe) y que ningún préstamo con ese ISBN esté activo (devuelto == False). Si las condiciones se cumplen, elimina la clave del catálogo y persiste el estado.

buscar_libros(catalogo, termino)

Recorre todos los libros del catálogo y retorna una lista con aquellos cuyo título, autor o género contienen el término de búsqueda. La comparación se hace normalizando acentos y usando minúsculas. Lanza una excepción si ningún libro coincide.

registrar_usuario(usuarios, numero_socio, nombre)

Verifica que el número de socio no exista. Crea el diccionario Usuario con prestamos_activos vacío, lo inserta en usuarios y persiste.

dar_baja_usuario(usuarios, numero_socio)

Verifica que el socio exista y que su lista prestamos_activos esté vacía (no puede darse de baja con préstamos pendientes). Elimina la clave del diccionario y persiste.

historial_usuario(prestamos, numero_socio, usuarios)

Verifica que el usuario exista. Filtra la lista de préstamos y retorna todos aquellos cuyo numero_socio coincide, incluyendo préstamos activos y devueltos. Lanza excepción si el usuario no existe o no tiene historial.

registrar_prestamo(catalogo, usuarios, prestamos, numero_socio, isbn, fecha_prestamo)

Valida que el usuario exista, que el ISBN exista y que haya ejemplares disponibles. Calcula la fecha_limite sumando 7 días a la fecha_prestamo con timedelta. Crea el préstamo, decrementa ejemplares_disponibles, agrega el ISBN a prestamos_activos del usuario y persiste.

registrar_devolucion(usuarios, catalogo, prestamos, numero_socio, isbn)

Verifica que el usuario y el libro existan. Busca el préstamo activo del usuario para ese ISBN, lo marca como devuelto, incrementa ejemplares_disponibles y elimina el ISBN de prestamos_activos del usuario. Persiste los cambios. Si no encuentra un préstamo activo para ese caso, lanza excepción.

listar_vencidos(prestamos, fecha_actual)

Filtra los préstamos cuya fecha_limite es anterior a la fecha actual y que no han sido devueltos. Retorna la lista o lanza excepción si no hay vencidos.

guardar_datos(catalogo, usuarios, prestamos, ruta)

Abre el archivo JSON en UTF-8, actualiza solo las claves que recibe como argumento (las que son None se mantienen sin cambios) y reescribe el archivo. Captura las exceptions FileNotFoundError, JSONDecodeError, PermissionError y OSError.

cargar_datos(ruta)

Lee el archivo JSON en UTF-8 y retorna un DataBiblioteca con las tres estructuras. Lanza excepción descriptiva si el archivo no existe, no es JSON válido o no tiene permisos.

normalizar(palabra)

Reemplaza vocales con tilde (á, é, í, ó, ú) por sus equivalentes sin tilde. Utilizada junto con lower() para comparaciones de búsqueda insensibles a acentos.

recomendar_libros(catalogo, prestamos, usuarios, numero_socio, n)

Obtiene el historial del usuario y cuenta la frecuencia de géneros leídos. Ordena los géneros de mayor a menor frecuencia. Filtra libros del catálogo que pertenezcan a los tres géneros más leídos y que no hayan sido leídos por ese usuario. Retorna n recomendaciones.

Situaciones límites

Identificamos algunos escenarios donde la app se comporta de manera incongruente o deficiente, pero que se salen del scope de la actividad. Algunas de estas son:

- Al eliminar un usuario/socio sus préstamos devueltos persisten, si se crea otro nuevo socio usando ese N° de socio se le asignarían esos prestamos.